

✓ (72 points possible)

This assignment needs the file books2.csv, which should be at the same location where you got this file.

✓ Problem 1: AI Memory (16 pts [4, 7, 5])

An AI chatbot needs to remember the last \$k\$ exchanges between a human and itself. Each exchange consists of something a human said and something the AI said back, for example,

Human: How are you?

AI: I am an AI and have no feelings. Have a nice day!

(a, 4 pts) Define a simple Exchange class with a constructor `__init__(self, human, ai)` that initializes the two strings associated with the human and AI dialogue. Override `__str__(self)` so that it returns output that looks like the example above ("Human: " and "AI: " added, and the two lines concatenated with a newline `\n` in-between).

```
# TODO
class Exchange:
    def __init__(self, human, ai):
        self.human = human
        self.ai = ai

    def __str__(self):
        return f"Human: {self.human} \nAI: {self.ai}"
```

```
print(Exchange("Hi", "Hi!")) # expect "Human:  Hi\nAI:  Hi!"
```

```
Human: Hi
AI: Hi!
```

(b, 7 pts) Define a `WindowedMemory` class. The constructor should take a single argument k - this will define how many exchanges it can hold. It should also initialize a list `memories` to be the empty list. Then define a single method, `add_exchange(self, exchange)`. If there are fewer than k exchanges in the `memories` list, add the exchange to the end of that list. If there are k exchanges in the list already, drop the 0th element and then append to the end of the list. Finally, override `__str__()` to concatenate all the exchanges, ending each with a newline.

```
# TODO
```

```
class WindowedMemory:
    memory : list
    def __init__(self, k):
        self.memory = []*k

    def add_exchange(self, exchange):
        for i in range(len(self.memory)-1):
            self.memory[i+1]=str(self.memory[i])
            self.memory[i]= str(exchange)
    def __str__(self):
        retval : str = ""
        for i in self.memory:
            retval += i+"\n"
        return retval
```

```
memory = WindowedMemory(2)
memory.add_exchange(Exchange("Hi!", "Hi!"))
memory.add_exchange(Exchange("How are you?", "It's a good day!"))
memory.add_exchange(Exchange("Try not to take over the world!", "I ob:
print(memory) # should have forgotten Hi exchange, remember the rest
```

```
"Human: Try not to take over the world!
AI: I object to your generalization!"
"Human: How are you?
AI: It's a good day!"
```

c, 5 pts) Maybe we want a window buffer, maybe we don't; every year AI is getting better at making use of more and more memory. In the space below, define two classes, Memory and WindowedMemory. The only difference between the two should be that WindowedMemory has a maximum number of exchanges it can remember, while Memory is unlimited (and has no k parameter in the constructor). Make use of inheritance, being sure you're following the convention that A inherits from B if A "is-a" B. You can copy code from above, but you should decide which of the classes the code goes into for the best object-oriented design.

```
# TODO
class Memory:
    memory : list
    def __init__(self):
        self.memory = []

    def add_exchange(self, exchange):
        self.memory.append(str(exchange))
    def __str__(self):
        retval : str = ""
        for i in self.memory:
            retval += i+"\n"
        return retval
```

```
# The previous WindowedMemory test should still work, but here's Memo
memory = Memory()
memory.add_exchange(Exchange("Hi!", "Hi!"))
memory.add_exchange(Exchange("How are you?", "It's a good day!"))
memory.add_exchange(Exchange("Try not to take over the world!", "I ob
print(memory) # should remember all of it
```

```
Human: Hi!
AI: Hi!
Human: How are you?
AI: It's a good day!
Human: Try not to take over the world!
AI: I object to your generalization!
```

Problem 2: The Book Class (42 points

[2,3,3,3,4,7,8,7,5])

a) Create a Book class with the following methods:

- (2 pts) A constructor that takes the author (string), title (string), and a list of ratings (integers)
- (3 pts) A `__str__()` method that returns the string "[title] by [author]" when invoked
- (3 pts) An `average_rating()` method that takes no argument and returns the average of all the ratings in the list
- (3 pts) A `rate()` method that takes one integer argument and adds that integer to the end of the ratings, returning the new list of ratings

```
# TODO Book class with __init__(), __str__(), average_rating(), rate()
class Book():
    author : str
    title : str
    ratings : list[int]

    def __init__(self,auth:str,tit:str,rat:list[int]):
        self.author = auth
        self.title = tit
        self.ratings = rat

    def __str__(self):
        return f"{self.title} by {self.author}"

    def average_rating(self):
        retval = 0
        for i in self.ratings:
            retval+= i
        return retval/len(self.ratings)

    def rate(self,nr:int):
        self.ratings.append(nr)
        return self.ratings
```

```
# Tests
my_book = Book('Mary Shelley', 'Frankenstein', [5, 4, 3, 5, 3])
print(my_book) # expect 'Frankenstein by Mary Shelley'
print(my_book.average_rating()) # expect 4.0
print(my_book.rate(5)) # expect [5, 4, 3, 5, 3, 5]
```

```
Frankenstein by Mary Shelley
4.0
[5, 4, 3, 5, 3, 5]
```

b, 4 pts) Sometimes data for an entity can be spread across multiple records in the dataset. Code the function `merge()` which takes two `Book` objects as arguments, and returns a single `Book` record where the author and title match the first argument, and the ratings are a concatenation of the two ratings lists.

```
# TODO merge()
def merge(b1 : Book, b2 : Book):
    return Book(b1.author,b1.title,b1.ratings+b2.ratings)
```

```
book1 = Book('Jane Austen', 'Pride and Prejudice', [5, 4, 5, 3])
book2 = Book('J. Austen', 'Pride and Prejudice', [5, 5])
book3 = merge(book1,book2)
print(book3) # Expect Pride and Prejudice by Jane Austen
print(book3.ratings) # Expect [5, 4, 5, 3, 5, 5]
```

```
Pride and Prejudice by Jane Austen
[5, 4, 5, 3, 5, 5]
```

c, 7 pts) Code a class that inherits from `Book`, `Nonfiction`. Give it a constructor identical to `Book`'s, except it has one additional argument: the edition number (an int). Then add to the `Nonfiction` class a method `more_recent_than()` which takes a `Book` as an argument and returns true if 1) the authors and titles match, *and* 2) the `Book` calling it has a higher edition number than the argument.

```
# TODO Nonfiction class with __init__(), more_recent_than()
class Nonfiction(Book):
    edition : int
    def __init__(self, auth, tit, rat, edit):
        super().__init__(auth, tit, rat)
        self.edition = edit

    def more_recent_than(self, other : Nonfiction):
        return self.edition > other.edition
```

```
book1 = Nonfiction('Feynman', 'Physics', [5, 4, 5], 1)
book2 = Nonfiction('Feynman', 'Physics', [5, 5, 5], 2)
book3 = Nonfiction('Herschbach', 'Chemistry', [5, 5, 5], 1)

print(book2.more_recent_than(book1)) # expect True
print(book2.more_recent_than(book3)) # expect False, author/title mismatch
print(book3.average_rating()) # Should have inherited this method from
```

```
True
True
5.0
```

d, 8pts) Code a class Fiction that inherits from Book and has an extra attribute *sequels*, which should be a list of Fiction objects (or empty if there is no sequel). Accept an extra argument in the constructor to set this attribute. Throw a ValueError if any of the elements of the list are not Fiction objects.

Then write a method `this_and_rest()` that returns a string that lists the book itself followed by all the books in its sequels list. The books should be turned to strings using the Book `__str__()` method, implicitly or explicitly. Separate the books after the first with commas and spaces - see the example below. (You may recall you can use "self" to refer to the object itself.)

```
# TODO Fiction class with __init__(), this_and_rest()
from __future__ import annotations

class Fiction(Book):
    sequels : list[Fiction]

    def __init__(self, auth, tit, rat, seq : list[Fiction]):
        if not all(isinstance(s, Fiction) for s in seq):
            raise TypeError("All sequels must be Fiction Type")
        super().__init__(auth, tit, rat)
        self.sequels = seq

    def this_and_rest(self):
        retval = str(self) + ", "
        for i in self.sequels:
            retval+= str(i) + ", "
        return retval
```

```
# Test, should throw an error because sequel isn't Fiction
book1 = Nonfiction('Feynman', 'Physics', [5, 4, 5], 1)
book2 = Fiction('Mary Shelley', 'Frankenstein', [5, 5, 5], [book1])
```

```
-----
-----
TypeError                                Traceback (most recent call
last)
Cell In[42], line 3
      1 # Test, should throw an error because sequel isn't Fiction
      2 book1 = Nonfiction('Feynman', 'Physics', [5, 4, 5], 1)
----> 3 book2 = Fiction('Mary Shelley', 'Frankenstein', [5, 5, 5],
[book1])

Cell In[41], line 9, in Fiction.__init__(self, auth, tit, rat, seq)
      7 def __init__(self, auth, tit, rat, seq : list[Fiction]):
      8     if not all(isinstance(s, Fiction) for s in seq):
----> 9         raise TypeError("All sequels must be Fiction Type")
     10     super().__init__(auth, tit, rat)
     11     self.sequels = seq
```

```

# Test this_and_rest()
king = Fiction('J.R.R. Tolkien', 'The Return of the King', [], [])
towers = Fiction('J.R.R. Tolkien', 'The Two Towers', [], [king])
fellowship = Fiction('J.R.R. Tolkien', 'The Fellowship of the Ring',
hobbit = Fiction('J.R.R. Tolkien', 'The Hobbit', [], [fellowship, towers])

print(hobbit.this_and_rest())
# """ Should be:
# The Hobbit by J.R.R. Tolkien, The Fellowship of the Ring by J.R.R. Tolkien,
# The Two Towers by J.R.R. Tolkien, The Return of the King by J.R.R. Tolkien
# """

```

```

The Hobbit by J.R.R. Tolkien, The Fellowship of the Ring by J.R.R. Tolkien,
The Two Towers by J.R.R. Tolkien, The Return of the King by J.R.R. Tolkien

```

e, 7 pts) Create a new BookDict class. The argument for the constructor should be the string name of a csv file. The csv columns in order from left to right are title, author, and a comma-separated string of ratings. The constructor should create a dictionary from title (string) to the Book object with that title. Call that dictionary my_dict and make it a field (attribute) of the BookDict object. (You can assume titles are unique for this exercise. Note that you will need to use the function float() to convert string ratings to floating point numbers.)

f, 5 pts) The one other method for the BookDict, lookup(), should take a title (string) as an argument, and print the information stored in the Book for that title. The printed information should follow the format '[title] by [author] ([average_rating])' -- for example, 'Pride and Prejudice by Jane Austen (4.2)'. Print no more than two digits after the decimal place. lookup() should also return the Book, besides printing its information. If the title isn't in the dictionary, print 'Not Found' and return None. Use methods you have already created when you can. You can again assume titles are unique.

```
# TODO BookDict class with __init__(), lookup()
import csv

class BookDict:
    my_dict : dict
    def __init__(self, csv_filename: str):
        self.my_dict = {} # title → Book
        with open(csv_filename, newline='', encoding='utf-8') as file:
            reader = csv.reader(file)
            for row in reader:
                if not row:
                    continue
                title, author, ratings_str = row[0], row[1], row[2]
                ratings = [float(r.strip()) for r in ratings_str.split()]
                self.my_dict[title] = Book(author, title, ratings)

    def lookup(self, title: str):
        book = self.my_dict.get(title)
        if book:
            print(book)
            return book
        else:
            print("Not Found")
            return None
```

```
# from google.colab import files
# im using VSCODE

# files.upload() # Upload books2.csv
```

```
# Test: should print 'Pride and Prejudice by Jane Austen (4.2)'
my_bookdict = BookDict('books2.csv')
my_book = my_bookdict.lookup('Pride and Prejudice')
# Test: should print 'Not Found'
book2 = my_bookdict.lookup('Erehwon')
```

```
Pride and Prejudice by Jane Austen
Not Found
```

✓ Problem 3: Recursion (12 points [4,4,4])

a, 4 pts) Suppose some code takes a number of steps $f(N)$ for input size N equal to $f(N-1) + f(N-3) + 1$. $f(0)$, $f(1)$, and $f(2)$ are all 100, making $f(3) = f(2) + f(0) + 1 = 201$. Write a function `running_time(N)` that recursively computes the number of steps required for input size N .

```
# TODO
```

```
def running_time(i:int):  
    if i <= 2:  
        return 100  
    return running_time(i-1) + running_time(i-3) + 1
```

```
print(running_time(3)) # Expect 201  
print(running_time(10)) # Expect 2827
```

```
201  
2827
```

b, 4 pts) Write a recursive function `recursive_lg()` that computes the log base 2 of the input, rounded up. You can compute this by counting how many divisions by 2 you need to do to get a number that is 1 or less; but you need to do this recursively.

```
# TODO  
import math  
def recursive_lg(i:int):  
    if i==1:  
        return 0  
    elif i<2:  
        return math.ceil(i/2)  
    return 1 + recursive_lg(i/2)
```

```
# Tests  
print(recursive_lg(1)) # Expect 0  
print(recursive_lg(4)) # Expect 2  
print(recursive_lg(6)) # Expect 3  
print(recursive_lg(1025)) # Expect 11
```

```
0  
2  
3  
11
```

c, 4 pts) The iterated logarithm, sometimes written $\log^*(n)$, is the number of times you need to take the log base 2 to get 1. (This function grows very slowly and is usually 5 or less; see https://en.wikipedia.org/wiki/Iterated_logarithm.) Write `recursive_log_star(n)` that recursively computes this number.

```
# TODO
def recursive_log_star(i :int):
    if(i==1):
        return 0
    return 1 + recursive_log_star(recursive_lg(i))
```

```
print(recursive_log_star(1)) # Expect 0
print(recursive_log_star(65536)) # Expect 4
print(recursive_log_star(2 ** 65536)) # Expect 5
#3rd test crashed
```

```
0
4
-----
OverflowError                                Traceback (most recent call
last)
Cell In[71], line 3
      1 print(recursive_log_star(1)) # Expect 0
      2 print(recursive_log_star(65536)) # Expect 4
----> 3 print(recursive_log_star(2 ** 65536)) # Expect 5

Cell In[70], line 5, in recursive_log_star(i)
      3 if(i==1):
      4     return 0
----> 5 return 1 + recursive_log_star(recursive_lg(i))

Cell In[65], line 8, in recursive_lg(i)
      6 elif i<2:
      7     return math.ceil(i/2)
----> 8 return 1 + recursive_lg(i/2)

OverflowError: integer division result too large for a float
```

✓ AI Statement (2 points)

Please put an X in the box that best matches your AI use for this homework ([X] in the markup will become a checked box). There is no penalty for using AI, but we want to understand how you're using it.

- ☐ I did not use AI to complete this assignment.
- ☒ I didn't use AI to debug or generate code, but I did use it as a Python reference.
- ☐ I didn't use AI to generate code, but I did use it to debug some code.
- ☐ I did use AI to generate some code, but not all the code for any problem.
- ☐ I used AI to solve at least one problem completely.

When you are done, submit both your .ipynb (File->Download->Download .ipynb in Google Colab) and a PDF (File->Print->Save to PDF) to Blackboard where you found this assignment. (The backup PDF helps us give you points if there's a problem with your .ipynb.)

Start coding or generate with AI.

